

Guess Market - Exercise 3

Name: Dan Alushe

ID: 212472864

Email: danalushe15@gmail.com

GitHub: <https://github.com/danal15/guess-market>

Bonus implemented

Chat. Every logged in user can open the Chat panel at the bottom of the window and talk to everyone else. The messages go through the server and arrive with the normal refresh. Only new messages are sent each time.

How to run

- Put guess-market.war in tomcat\webapps and start Tomcat. It deploys under /guess-market.
- Go into the ex3-client folder and double click run.bat (or run: java -jar guess-market-client.jar). You can open several clients at the same time.
- The client uses <http://localhost:8080/guess-market> by default. A different server address can be passed to run.bat.
- Tomcat 10.1 and Java 25 are required. JavaFX and Gson are already packed in the files.
- Log in with any name. There are no passwords. A name that is already in use is refused.
- A new user starts with \$0. Use "Load funds..." on the Account tab to add money.
- Use "Upload Events File..." on the Account tab to load an exercise 3 file. The user who uploads it becomes the MM of its events.

How to use it

- Events tab - view only. Shows all events in the system with filters by method / status / commission. Clicking an event shows its details, trades and chart.
- Account tab - the user's own screen. It shows the balance, money movements, other users, file upload and the actions that can be done on each event.
- Other users' account movements are private. Their name, balance and whether they are an MM are visible. Orders in the order book are public.
- Everything refreshes automatically once a second, so changes from other users appear without pressing anything.

Project structure

Four modules: engine and desktop are from exercise 2, and server and client are new.

engine - the system logic. It has no JavaFX or servlet code. It also stores the logged in users, money movements and chat.

server - the WAR with the servlets. The servlets call the engine and return JSON. EngineHolder creates the one shared engine and GmServlet has the common servlet code.

client - the JavaFX client. MarketClient implements GMEngine, but its calls go to the server over HTTP. Transport handles the HTTP calls and the session cookie.

desktop - the exercise 2 UI code. The client reuses the event details, trade forms, dialogs, charts, skins and stylesheets from it.

Main new classes

SnapshotDTO - contains the information needed for one client refresh.

Movement / MovementDTO - one movement of money in a user account.

EngineHolder - creates and stores the shared engine used by all servlets.

GmServlet - base servlet with the common JSON, session and error handling.

Transport / MarketClient - the client side. MarketClient implements GMEngine and Transport does the HTTP communication.

ClientRootController - controls the two tabs and the automatic refresh.

AccountTabController - controls the Account tab.

MarketFileLoader.loadInto - loads exercise 3 files into a market that is already running.

Decisions I made

- A new user starts with \$0 and can add money with "Load funds".
- If a user goes below zero they are blocked, like in exercise 2. Loading enough funds removes the block.
- Events are identified by name. Two events cannot have the same name, including names that only differ by case or spaces.
- An uploaded file is loaded completely or not at all. If one event is invalid, nothing from the file is added.
- Two events with the same name inside one file are also refused.
- Exercise 3 files cannot contain a users section. An exercise 2 file is refused with a message.
- Uploaded files are parsed from the request and are not saved on the server.
- Events must have exactly two options.
- The current user is taken from the session, not from values sent by the client.
- The client refreshes once a second. Chat only sends messages that the user has not received yet.
- Only one refresh request runs at a time.
- The engine public methods are synchronized because several clients can use the system at the same time.
- Event filters are applied in the client.
- If the server is unavailable, the client shows a message in the top bar and keeps trying.
- Nothing is saved when Tomcat stops.
- The LMSR logic, order book matching and minting, commissions and event closing are unchanged from exercise 2.